

Code-Layout, Readability and Reusability

Date	30 June 2026
Team ID	
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Standard Python and HTML/CSS indentation followed.
2	Proper File Structure	Files and folders are logically organized	Yes	Separate folders for templates, static files, models, and dataset.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables such as temperature, humidity, rainfall, model, and scaler are self-explanatory.
4	Function / Method Names	Functions are descriptively named	Yes	Functions like predict(), home(), and run_training_pipeline() clearly describe their purpose.
5	Code Comments	Inline and block comments explain logic	Yes	Comments explain preprocessing, model loading, prediction flow, and validation logic.
6	Modular Design	Code is split into reusable functions/modules	Yes	Separate modules for training, prediction, frontend, and model storage improve maintainability.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Code is organized to minimize duplication and improve readability.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Input validation and exception handling are implemented in the Flask backend to manage invalid requests.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Machine Learning Prediction Model	Python, Scikit-learn, Joblib	Flask Backend Prediction	High
2	Data Preprocessing Module	Python, Pandas,	Model Training Pipeline	High

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
		NumPy		
3	Flask API Routes	Python, Flask	Home Page & Prediction Requests	High
4	Frontend Form Components	HTML, CSS, JavaScript	User Input & Result Display	Medium
5	Input Validation Module	Python, Flask	Prediction Request Processing	High

Overall Code Quality Assessment:

Aspect	Rating (1–5)	Comments
Code Layout & Structure	5	Well-organized project structure with separate frontend, backend, dataset, and model directories.
Readability	5	Meaningful naming conventions, proper formatting, and clean code improve readability.
Reusability	5	Modular architecture enables reuse of preprocessing, prediction, and API components.
Documentation / Comments	4	Adequate comments are included to explain important logic and workflows.
Overall Score	4.8 / 5	The codebase is well-structured, maintainable, reusable, and suitable for future enhancements such as cloud deployment and weather API integration.